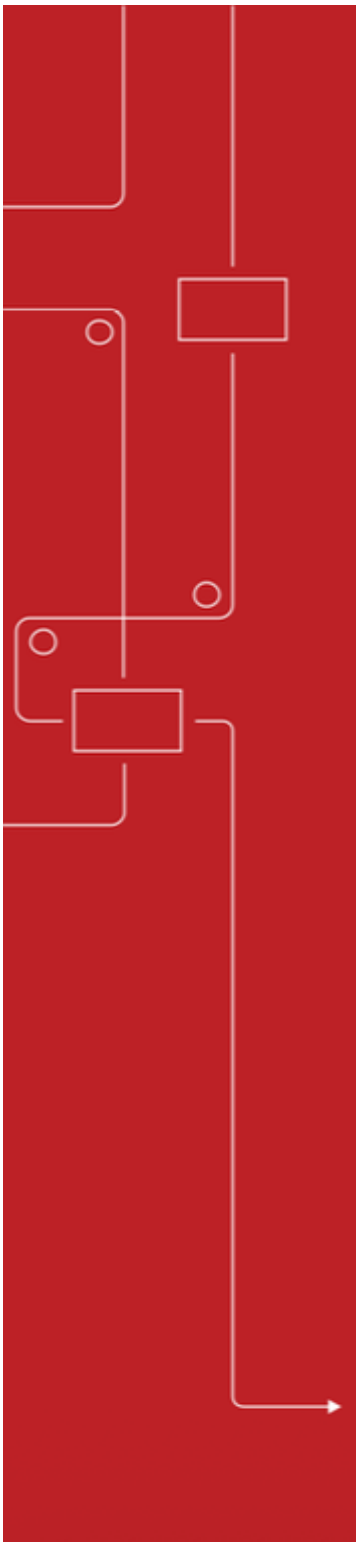


Red Hat
Training and
Certification

Administering RHEL in SELinux MLS Mode

Red Hat Enterprise Linux 10.0

Student Workbook
Edition 1

Administering RHEL in SELinux MLS Mode

Red Hat Enterprise Linux 10.0

Edition 1

Edition date 2026-09-01

Publication date 2026-09-01

Author: Jesse Scott

Course Architect: Rob Locke

© 2026 Red Hat, Inc.

The contents of this course and all its modules and related materials, including handouts to audience members, are © 2026 Red Hat, Inc.

No part of this publication may be stored in a retrieval system, transmitted or reproduced in any way, including, but not limited to, photocopy, photograph, magnetic, electronic or other record, without the prior written permission of Red Hat, Inc.

This instructional program, including all material provided herein, is supplied without any guarantees from Red Hat, Inc. Red Hat, Inc. assumes no liability for damages or legal action arising from the use or misuse of contents or details contained herein.

If you believe Red Hat training materials are being used, copied, or otherwise improperly distributed, please send email to training@redhat.com or phone toll-free (USA) +1 (866) 626-2994 or +1 (919) 754-3700.

Red Hat, Red Hat Enterprise Linux, the Red Hat logo, JBoss, OpenShift, Fedora, Hibernate, Ansible, RHCA, RHCE, RHCSA, Ceph, and Gluster are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

Linux® is the registered trademark of Linus Torvalds in the United States and other countries.

All other trademarks are the property of their respective owners.

1. Foundations of Multi-Level Security (MLS)

Objective: Understand the theoretical underpinnings of MLS and how it differs from standard Linux security models.

Introduction to Access Control Models

Over time, there have been a number of security models in use by operating systems. Access control models such as Discretionary Access Control (DAC), Mandatory Access Control (MAC), and Multi-Level Security (MLS) are examples covered in this section. An access control model is a security framework that defines how users and systems get permission to view, use, or change digital and physical resources.

Discretionary Access Control (DAC) vs. Mandatory Access Control (MAC)

Discretionary Access Control (DAC) is the security model that is most used by conventional operating systems. This model enforces security by ownership. So, if a user owns a file, that user is able to set read, write, and execute permissions for the file. With DAC, the users have more control over the system, because they manage the data at their discretion.

One major concern presented by the DAC model in Linux is the risk that is present in having a root account. Since the super-user is able to control all files and processes, the user can see and do just about anything on the system. Should the root account be compromised, a bad actor could take over the system and all of its data. Because of this, it would be more secure to limit the use of a root account, while moving the control from users to the owner of the system.

Mandatory Access Control (MAC) focuses control on the owner of the system and makes security policy enforcement mandatory. As security policies are implemented by system administrators, users are not able to override them, even if they have elevated privileges. In this way, the file and process protection is independent of the system users.

The Bell-LaPadula Confidentiality Model

Multi-Level Security (MLS) protects files and processes by using data sensitivity levels based on the Bell-LaPadula Model (BLP). Using the “no read up, no write down” principle, this model prevents unauthorized access to highly sensitive information. BLP specifies how information is

able to flow within the system based on labels that are attached to each subject and object. Because of this, users are only able to read files at their own sensitivity level and lower. Also, with this method, data can only flow from lower levels to higher levels, and never the other way around. The implementation of MLS on RHEL, actually applies a modified version of BLP called Bell-La Padula with write equality. In this modified model, users can read files at their own sensitivity level and lower, but can write only at exactly their own level.

SELinux Policies Compared

SELinux (Security-Enhanced Linux) is a security feature built into the Linux kernel that adds Mandatory Access Control (MAC). It uses a default-deny approach and security labels to strictly control what actions users and programs can perform, protecting the system even if a process is compromised.

SELinux policies differ primarily by scope and strictness, with targeted, strict, and MLS representing standard configurations. Targeted confines only specific network and boot services, strict confines nearly all system processes and users, and MLS enforces multi-level security clearance rules.

Targeted, Strict, and MLS

The targeted policy protects selected network daemons and system services. In this configuration, general user login sessions and standard shell processes run in an unconfined domain (unconfined_t). This is typically the default and ideal choice for standard servers and desktops, offering security for high-risk entry points without breaking regular user workflows.

The strict policy applies confinement rules universally across the entire operating system. In this configuration, every process, administrator action, and logged-in user is strictly confined by security policies. This is best for high-security environments where absolute control over every user and application action is mandatory.

The MLS (Multi-Level Security) policy implements extreme hierarchical clearance checks based on sensitivity labels (e.g., Secret, Top Secret) and categories. In this configuration, data flow is enforced by a modified Bell-LaPadula model where users can read down to lower levels but can only write at their exact clearance level. This is most often used in specialized government, military, or defense systems requiring strict data separation across security classifications.

The MLS Security Context Anatomy

Multi-Level Security (MLS) technology establishes a hierarchical data classification framework through sensitivity levels, typically including designations such as:

- Unclassified (lowest level of sensitivity)
- Confidential (low level of sensitivity)
- Secret (high level of sensitivity)
- Top Secret (highest level of sensitivity)

The standard SELinux MLS policy implementation provides 16 distinct sensitivity levels:

- The **s0** level represents minimum sensitivity.
- The **s15** level represents maximum sensitivity.

Specific terminology is utilized within MLS to categorize system components by sensitivity:

- Active entities like users and processes are defined as **subjects**, with their assigned sensitivity known as **clearance**.
- Passive system elements, including files and devices, are defined as **objects**, with their assigned sensitivity known as **classification**.

In SELinux, a Multi-Level Security (MLS) security context appends a hierarchical sensitivity level and non-hierarchical category compartments to the standard Type Enforcement label. Type enforcement in Multi-Level Security (MLS) is an access control method that labels system processes as subjects and files or devices as objects to strictly regulate interactions based on security domains. It acts as the core foundational layer for mandatory security policies. This strengthens security by enforcing strict information-flow controls based on the Bell-LaPadula model.

Understanding the Fourth Field

The fourth field of a security context (user:role:type:level) is the Level field, which defines the Multi-Level Security (MLS) attributes. This field is responsible for enforcing strict, data-classification boundaries. It dictates exactly what degree of data classification a process can interact with, formatted as a single level or a clearance range. This field is constructed using two distinct components: Sensitivity (Levels) and Categories (Compartments).

Sensitivities (Levels)

In SELinux Multi-Level Security (MLS) mode, sensitivity levels form a hierarchical ranking system from s0 (least sensitive) to s15 (most sensitive), controlling access via strict "no read-up, no write-down" information flow rules. The levels build upward, where a higher number provides access to more sensitive information. Standard policies configure 16 distinct levels (s0 through s15). Dominance rules are defined via policy statements to dictate which clearance level supersedes another during access checks.

Categories (Compartments)

In SELinux Multi-Level Security (MLS) mode, categories (also known as compartments) are non-hierarchical, unordered labels used to group or isolate data based on specific projects, departments, or organizations. While sensitivity levels control hierarchical access (e.g., Secret, Top Secret), categories provide horizontal security boundaries to enforce the principle of least privilege

Categories are defined as c0 through c1023 in traditional systems (supporting up to 1024 distinct categories). These appear in the fourth field of the SELinux security context string, attached directly to the sensitivity level using a colon (e.g., user:role:type:sensitivity:categories)

2.1. Preparing and Enabling MLS Mode on RHEL

Objective: Safely transition a RHEL system from standard targeted mode to MLS mode without locking yourself out.

Pre-installation Requirements & Risk Assessment

Using MLS is complex and does not map well to typical use cases. MLS is best suited for government and military organizations that need to protect classified information, but it could introduce challenges on most typical computer systems.

Avoid deploying the MLS policy on any system utilizing the X Window System. Additionally, during the filesystem relabeling process with MLS attributes, the system may block access to confined domains, potentially obstructing a successful boot. To mitigate this risk, you must transition SELinux into permissive mode prior to initiating the relabeling procedure. It is common to encounter numerous SELinux denials upon switching to MLS, many of which require advanced troubleshooting to resolve.

Why MLS can break standard applications and services

Multi-Level Security (MLS) breaks standard applications because it removes unconfined user domains, enforces strict sensitivity labels, and blocks cross-level data flows. Standard software lacks the required sensitivity tags and explicit clearance rules to function under MLS constraints.

Some core reasons for application breakage are as follows:

No Unconfined Domain: The standard targeted policy lets regular administrative tasks and root run without strict limits, but MLS strips out unconfined fallback roles entirely.

Bell-LaPadula Constraints: MLS enforces a strict “no read-up, no write-down” rule (with modified write equality), meaning standard apps cannot read configuration files or write logs that exist at different sensitivity ranges.

Missing Label Metadata: Standard files and process daemons lack hierarchical clearance tags (like s0-s0:c0.c1023), causing the kernel to deny access by default.

The Conversion Process

Change the active SELinux policy from the standard targeted configuration to the Multi-Level Security (MLS) model. It is critical to first shift into permissive mode for the filesystem relabeling phase to mitigate the risk of catastrophic boot failures.

Installing the MLS Policy Package

Install the selinux-policy-mls package:

```
# dnf install selinux-policy-mls
```

Modifying /etc/selinux/config

Use a text editor to modify the /etc/selinux/config configuration file:

```
# vi /etc/selinux/config
```

Change SELinux mode from enforcing to permissive and switch from the targeted policy to MLS:

```
SELINUX=permissive
SELINUXTYPE=mls
```

Save the changes, and close the editor.

Relabeling the filesystem for MLS

Prior to activating the Multi-Level Security (MLS) policy, it is essential to perform a full filesystem relabel to assign appropriate hierarchical security attributes to every system object:

```
# fixfiles -F onboot

System relabels on next boot
```

Reboot the system:

```
# systemctl reboot
```

Check for SELinux denials and switch SELinux back to enforcing mode

Check for any SELinux denials:

```
# ausearch -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts \
> recent -i
```

Since the preceding command may not address every situation, refer to Troubleshooting problems related to SELinux for comprehensive instructions on detecting, examining, and resolving SELinux denials.

Once you have verified that your system is free of SELinux-related issues, update the configuration in /etc/selinux/config to toggle SELinux back to enforcing mode:

```
SELINUX=enforcing
```

Reboot the system:

```
# systemctl reboot
```

Troubleshooting early boot failures

Should your system fail to boot or if you cannot log in after transitioning to MLS, append the `enforcing=0` parameter to the kernel command line. For additional guidance, refer to Changing SELinux modes at boot time.

Additionally, note that under MLS, SSH logins as the `root` user mapped to the `sysadm_r` SELinux role operate differently than logging in as `root` in `staff_r`. Prior to booting into MLS for the first time, consider allowing SSH logins for `sysadm_r` by setting the `ssh_sysadm_login` SELinux boolean to `1`. To enable `ssh_sysadm_login` while already running in MLS, log in as `root` in `staff_r`, change to the `root` user in `sysadm_r` by executing the `newrole -r sysadm_r` command, and then set the boolean value to `1`.

First Boot in MLS Mode

Verifying MLS status

Once the MLS conversion tasks are complete, you should confirm that SELinux has successfully changed over to enforcing mode. Furthermore, ensure that the active SELinux policy type reports as mls.

Confirm that SELinux is operating in enforcing mode:

```
# getenforce
```

```
Enforcing
```

Verify that the SELinux status shows that the mls policy is loaded:

```
# sestatus | grep mls
Loaded policy name:             mls
```

2.2. Guided Exercise: Preparing and Enabling MLS Mode on RHEL

In this exercise, you will prepare and enable MLS mode on a RHEL server.

1. In the terminal on the student workstation, make an SSH connection to servera.

```
student@workstation:~$ ssh student@servera
```

2. Elevate privileges to become the root user.

```
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

3. Install the selinux-policy-mls package.

```
[root@servera ~]# dnf install selinux-policy-mls
...output omitted...
Is this ok [y/N]: y
...output omitted...
Installed:
  mctrans-3.8-1.el10.x86_64
  policycoreutils-newrole-3.8-1.el10.x86_64
  selinux-policy-mls-40.13.26-1.el10.noarch
Complete!
```

4. Change the SELinux mode from enforcing to permissive and switch from the targeted policy to MLS.

4.1. Edit and save the /etc/selinux/config file to change the SELINUX parameter to permissive and the SELINUXTYPE parameter to mls.

```
# This file controls the state of SELinux on the system.
...output omitted...
SELINUX=permissive
...output omitted...
SELINUXTYPE=mls
...output omitted...
```

4.2. Relabel the filesystem.

```
[root@servera ~]# fixfiles -F onboot
System will relabel on next boot
```

4.3. Reboot servera.

```
[root@servera ~]# systemctl reboot
...output omitted...
The system will reboot now!
```

5. After giving some time for the reboot to complete, re-establish the SSH connection to servera.

```
student@workstation:~$ ssh student@servera
```

6. Elevate privileges to become the root user.

```
[student@servera ~]$ sudo -i
[sudo] password for student: student
[root@servera ~]#
```

7. Investigate any SELinux denials. Take note of any interesting output, but for now you are not expected to fix any denials.

```
[root@servera ~]$ ausearch -m \
> AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent -i
```

8. Create the ***admin*** user with ***redhat*** as password.

```
[root@servera ~]# useradd admin
[root@servera ~]# passwd admin
New password: redhat
...output omitted...
Retype new password: redhat
passwd: password updated successfully
```

9. Add the user ***admin*** to the ***wheel*** group for sudo access.

```
[root@servera ~]# usermod -aG wheel admin
```

10. Update configuration for the ***wheel*** group in the /etc/sudoers configuration file, so that TYPE=sysadm_t and ROLE=sysadm_r

10.1. Use the visudo command to edit the sudoers configuration file.

```
[root@servera ~]# visudo
```

10.2 Edit and save the sudoers configuration file, so that the configuration for the wheel group includes TYPE=sysadm_t and ROLE=sysadm_r

```
## Sudoers allows particular users to run various commands as
## the root user, without needing the root password.
...output omitted...

## Allows people in group wheel to run all commands
%wheel ALL=(ALL) TYPE=sysadm_t ROLE=sysadm_r ALL

...ouput omitted...
```

11. Map the ***staff_u*** SELinux user to the ***admin*** Linux user.

```
[root@servera ~]# semanage login -a -s staff_u -r \
"s0-s15:c0.c1023" admin
```

12. Restore file security contexts to default.

```
[root@servera ~]# restorecon -RFv /home/admin
Relabeled ...output omitted...
```

13. Temporarily set SELinux to enforcing to test access.

```
[root@servera ~]# setenforce 1
```

14. Connect to servera via SSH as the ***admin*** user to test access.

14.1. Disconnect from the current connection to servera.

```
[root@servera ~]# exit
logout
[student@servera ~]$ exit
```

```
student@workstation:~$
```

14.2. Connect via SSH as the admin user to servera.

```
student@workstation:$ ssh admin@servera
```

14.3. Elevate privileges to become the root user.

```
[admin@servera ~]$ sudo -i
[sudo] password for admin: redhat
[root@servera ~]#
```

15. Set SELinux persistently to enforcing in the /etc/selinux/config file.

15.1. Edit and save the /etc/selinux/config file to set SELINUX=enforcing.

```
# This file controls the state of SELinux on the system.
...output omitted...
SELINUX=enforcing
...output omitted...
SELINUXTYPE=mls
...output omitted...
```

15.2. Reboot the servera system.

```
[root@servera ~]# systemctl reboot
...output omitted...
The system will reboot now!
```

16. After giving some time for the reboot to complete, re-establish the SSH connection to servera.

16.1. SSH to servera as the ***admin*** user.

```
student@workstation:~$ ssh admin@servera
```

16.2. Elevate privileges to become the root user.

```
[admin@servera ~]$ sudo -i
[sudo] password for admin: redhat
```

```
[root@servera ~]#
```

17. Verify that MLS is properly configured.

    17.1. Use the getenforce command to check that SELinux is set to enforcing.

```
[root@servera ~]# getenforce
Enforcing
```

    17.2. Use the sestatus command to check that the SELinux type is set to mls.

```
[root@servera ~]# sestatus | grep mls
Loaded policy name:             mls
```

18. Disconnect from the SSH session to end the activity.

```
[root@servera ~]# exit
logout
[admin@servera ~]$ exit
student@workstation:~$
```

3.1. Managing Users, Clearance, and Categories

Objective: Configure Linux users with specific MLS clearance levels and compartments to enforce strict data separation.

Mapping Linux Users to SELinux Users

Mapping Linux users to SELinux Multi-Level Security (MLS) users links a standard system login to an SELinux identity and a sensitivity range like s0-s0:c0.c1023. This process uses the semanage login tool to enforce strict clearance levels and role restrictions.

The role of SEuser in MLS

In Security-Enhanced Linux (SELinux), an SEUser maps operating system users to specific SELinux policies, determining a user's authorized Multi-Level Security (MLS) range and roles.

Here is how the SEUser functions in an MLS configuration.:

The standard SELinux security context follows a four-part format of user:role:type:level. The SEUser dictates the outer boundaries of this identity ecosystem. Operating systems do not use Linux login names (like root or user1) directly in security choices. Instead, semanage login maps the Linux login to a backend SEUSer (such as user_u, staff_u, etc.). The SEUser controls which roles are reachable, and those roles control which types (domains) are reachable.

When an environment relies on the strict, hierarchical clearance structures of MLS, the SEUser acts as the gatekeeper for data confidentiality. To enforce clearances, each SEUSer is assigned a strict MLS clearance range (e.g., s0-s15:c0.c1023). A user cannot log into or transition to a security context that sits outside the MLS range mandated by their assigned SEUser.

To control information flow, the SEUser strictly limits a subject's ability to violate the Bell-La Padula model (which rules against "reading up" to higher classifications or "writing down" to lower levels.

The SEUser dictates access to specialized MLS administration roles. For instance, a standard user_u SEUser is blocked from sensitive data. Conversely, an administrative SEUSer might be

mapped to secadm_r (to change SELinux policy rules) or auditadm_r (to adjust auditing subsystems).

Assigning users to the staff_r, sysadm_r, or secadm_r roles

The staff_r role in an SELinux Multi-Level Security (MLS) policy environment provides standard, non-administrative user permissions with the unique capability to escalate privileges using sudo or su. This role permits file execution within the user's home directory and /tmp by default.

The sysadm_r role provides general system administration access, including managing system daemons, software, and regular operating tasks, while by default inheriting security policy management rights (secadm_r) unless explicitly separated.

The secadm_r role provides security administrator access to manage SELinux policy rules, security definitions, and object sensitivity levels. This role permits users to modify and load SELinux policy modules and rules. It also allows users to change file and user classification levels, as well as allows management of boolean values for MLS.

To assign a standard LInux user to the staff_r role, you must map the LInux user to the staff_u SELinux user, which has staff_r as its default role) and specify an MLS security range.

Run the following command to link a Linux user to the staff_r role.:

```
# semanage login -a -s staff_u -r "s0-s15:c0.c1023" username
```

Changing a user's SELinux identity requires resetting the context of their home directory files so they match the newly assigned staff_u ownership.:

```
# restorecon -FRv /home/username
```

To assign a standard Linux user to the sysadm_r role, you must map the Linux user to the sysadm_u or staff_u SELinux user (either of which is authorized to use the sysadm_r role) and specify the allowed MLS clearance range.

Run the following command to link a Linux user to the sysadm_r role.:

```
# semanage login -a -s sysadm_u -r "s0-s15:c0.c1023" username
```

Reset the context of the user's home directory.:

```
# restorecon -FRv /home/username
```

To assign a standard Linux user to the secadm_r role, you must modify the SELinux user to authorize the role and then map the Linux login to that SELinux user. Because secadm_r is a non-login role, users must first log in with a standard role (like staff_r) and transition into it.

To append secadm_r to the list of allowed roles for your chosen SELinux user (such as staff_u), run this command.:

```
# semanage user -m -R "staff_r system_r secadm_r" staff_u
```

After appending secadm_r to the list of allowed roles, map your specific Linux user to the modified SELinux user.:

```
# semanage login -a -s staff_u -r "s0-s15:c0.c1023" username
```

Note that because secadm_r cannot be used directly as an initial login context, the user must transition to it at runtime. You can achieve this by adding a rule to the /etc/sudoers file to automatically request the role context when using sudo to elevate privileges.

Configuring Clearance Levels

Using semanage login to define minimum and clearance levels

Following the activation of MLS, specific security clearance levels can be assigned to confined SELinux users. This controls their access by preventing them from reading objects with a higher sensitivity level or writing to files at different sensitivity levels.

By default, a user's assigned security clearance enforces the following restrictions:

- They cannot read objects that have a higher sensitivity level.
- They cannot write to objects at a different sensitivity level.

The procedure is as follows:

Optionally, switching to permissive SELinux mode can help prevent the introduction of errors into your policy and simplify troubleshooting.

```
# setenforce 0
```

Configure a clearance range for the SELinux user staff_u. For instance, the command sets a clearance range spanning from s1 to s15, where s1 serves as the default level of clearance.

```
# semanage user -m -L s1 -r s1-s15 staff_u
```

Generate SELinux file context rules for user home directories. The genhomedircon system script rebuilds the active SELinux policy and creates labels for each user home folder based on system password entries.

```
# genhomedircon
```

Restore the default file security contexts.

```
# restorecon -R -F -v /home/
```

Assign a clearance level to the user. In this example, s1 is the clearance level assigned.

```
# semanage login -m -r s1 example_user
```

Relabel the user's home directory to match the user's clearance level.

```
# chcon -R -l s1 /home/example_user
```

If you changed to permissive SELinux mode earlier, return to enforcing mode once you have verified that all operations work as intended.

```
# setenforce 1
```

Ensure the user is properly mapped to their corresponding SELinux user identity and has been allocated the appropriate clearance level.

```
# semanage login -l
Login Name      SELinux User    MLS/MCS Range        Service
__default__     user_u          s0-s0                *
example_user    user_u          s1                   *
```

To ensure the user's security level functions correctly, you can perform the following verifications:

- Confirm that the user is blocked from reading files with a higher level of sensitivity.

- Verify that the user is permitted to write to files sharing the exact same sensitivity.
- Ensure that the user is capable of reading files with a lower level of sensitivity.

Customizing Sensitivities and Categories

The mcstransd daemon and /etc/selinux/mls/setrans.conf work together to translate complex SELinux Multi-Level Security (MLS) sensitivity and category numbers into easy-to-read text names. mcstransd runs in the background and reads the configuration mapping file, letting users see clear labels instead of raw tags like s0:c0

Using mctransd to translate cryptic levels into human-readable labels

mctransd acts as a background service that listens for label translation requests. It connects with libselinux via a local socket (/var/run/setrans) to convert codes on the fly. This helps users and system tools better understand security contexts in system logs, audits, and commands.

Example sensitivity mappings are as follows.:

```
# Sensitivity Mappings (MLS System)
s0=Unclassified
s1=Confidential
s2=Secret
s3=Top Secret
```

Example category mappings are as follows.:

```
# Category Mappings (MCS System)
c0=Finance
c1=Human Resources
c2=Project Alpha
c3=Legal
c4=Medical Records
```

Configuring /etc/selinux/mls/setrans.conf

The configuration file, setrans.conf, stores the custom map between raw MLS/MMCS levels (such as s0:c0) and plain text names (such as Confidential). It supports sensitivity levels (s0 through s15) and categories (c0 through c1023). The file maps entries using a simple rule format, for example: s0:c0=Secret or s0:c1=Internal.

Below is an example configuration block for setrans.conf.:

```
# /etc/selinux/mls/setrans.conf
# Format: Raw_SELinux_Label=Human_Readable_Label

# --- Sensitivity Level Definitions ---
s0=Unclassified
s1=Confidential
s2=Secret
s3=Top Secret

# --- Category / Compartment Definitions ---
s0:c0=Unclassified:Finance
s0:c1=Unclassified:HR
s0:c2=Unclassified:Legal

# --- Multi-Category Combinations ---
s2:c0,c1=Secret:Finance,HR
s3:c0.c3=Top Secret:All-Depts
```

3.2. Guided Exercise: Managing Users and Clearance

In this exercise, you will create and manage users that can perform administrative functions in SELinux MLS mode.

1. Connect via SSH to servera as the admin user.

```
student@workstation:~$ ssh admin@servera
```

2. Elevate privileges to become the root user.

```
[admin@servera ~]$ sudo -i
[sudo] password for admin: redhat
[root@servera ~]#
```

3. List the available SELinux users and check the SELinux roles associated with each.

```
[root@servera ~]# semanage user -l
SELinux User ... SELinux Roles
...output omitted...
staff_u      ... system_r audit adm_r secadm_r staff_r sysadm_r
...output omitted...
sysadm_u     ... sysadm_r
```

4. Create a standard Linux user named secadmin:

```
[root@servera ~]# useradd secadmin
```

5. Set the password for the secadmin user to be redhat:

```
[root@servera ~]# passwd secadmin
New password: redhat
Retype new password: redhat
passwd: password updated successfully
```

6. Link the secadmin user to the staff_u SELinux user.:

```
[root@servera ~]# semanage login -a -s staff_u -r \
"s0-s15:c0.c1023" secadmin
```

7. Reset the context of the home directory files for the secadmin user so they match the newly assigned staff_u ownership.:

```
[root@servera ~]# restorecon -FRv /home/secadmin
```

8. Use the visudo command to edit the sudoers file.

```
[root@servera~]# visudo
```

9. Add the entry in bold to the sudoers configuration file (in the location shown) and then save.

```
...output omitted...
## Allow root to run any commands anywhere
root      ALL=(ALL)      ALL

## Allow secadmin to run commands as root under the secadm_r role
secadmin ALL=(ALL)  TYPE=secadm_t  ROLE=secadm_r  ALL

...output omitted...
```

10. List the active Linux user login mappings to check that the secadmin user is mapped to staff_u .:

```
[root@servera ~]# semanage login -l
Login Name     SELinux User    ...
...output omitted...
secadmin       staff_u         ...
```

11. Create a standard Linux user named sysadmin.

```
[root@servera ~]# useradd sysadmin
```

12. Add redhat as the password for the sysadmin user.

```
[root@servera ~]# passwd sysadmin
New password: redhat
```

```
Retype new password: redhat
passwd: password updated successfully
```

13. Link the sysadmin user to the staff_u SELinux user.

```
[root@servera ~]# semanage login -a -s staff_u -r \
"s0-s15:c0.c1023" sysadmin
```

14. Reset the context of the sysadmin user's home directory.

```
[root@servera ~]# restorecon -FRv /home/sysadmin
```

15. Use the visudo command to edit the sudoers file.

```
[root@servera~]# visudo
```

16. Add the entry in bold to the sudoers configuration file (in the location shown) and then save.

```
...output omitted...
## Allow root to run any commands anywhere
root      ALL=(ALL)      ALL

## Allow secadmin to run commands as root under the secadm_r role
secadmin ALL=(ALL)  TYPE=secadm_t  ROLE=secadm_r  ALL

## Allow sysadmin to run commands as root under the sysadm_r role
sysadmin ALL=(ALL) TYPE=sysadm_t   ROLE=sysadm_r  ALL

...output omitted...
```

17. List the active Linux user login mappings to check that the sysadmin user is mapped to staff_u .

```
[root@servera ~]# semanage login -l
Login Name     SELinux User    ...
...output omitted...
sysadmin       staff_u         ...
...output omitted...
```

18. Exit the ssh connection to servera.

```
[root@servera ~]# exit
exit
[admin@servera ~]$ exit
logout
Connection to servera closed
student@workstation:~$
```

19. Make an ssh connection to servera as the secadmin user.

```
student@workstation:~$ ssh secadmin@servera
```

20. Enter an interactive root session using sudo.

```
[secadmin@servera ~]$ sudo -i
[sudo] password for secadmin: redhat
```

21. Lookup the security context of the root session to check that the user is granted the secadm_r role.

```
[root@servera ~]# id -Z
staff_u:secadm_r:secadm_t:s0-s15:c0.c1023
```

22. Exit the ssh connection to servera.

```
[root@servera ~]# exit
exit
[secadmin@servera ~]$ exit
logout
Connection to servera closed
student@workstation:~$
```

23. Make an ssh connection to servera as the sysadmin user.

```
student@workstation:~$ ssh sysadmin@servera
```

24. Enter an interactive root session using sudo.

```
[sysadmin@servera ~]$ sudo -i
```

```
[sudo] password for sysadmin: redhat
```

25. Lookup the security context of the root session to check that the user is granted the sysadm_r role.

```
[root@servera ~]# id -Z
staff_u:sysadm_r:sysadm_t:s0-s15:c0.c1023
```

26. Exit the ssh connection to servera to end the activity

```
[root@servera ~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed
student@workstation:~$
```

3.3. Guided Exercise: Managing Categories with mcstrans

In this exercise, you will customize mapping of sensitivity and category labels by editing setrans.conf.

1. Connect via SSH to servera as the sysadmin user.

```
student@workstation:~$ ssh sysadmin@servera
```

2. Elevate privileges to become the root user.

```
[sysadmin@servera ~]$ sudo -i
[sudo] password for sysadmin: redhat
[root@servera ~]#
```

3. Check the status of the mcstrans service.

```
[root@servera ~]# systemctl status mcstrans
...output omitted...
     Active: inactive (dead)
...output omitted...
```

4. Edit /etc/selinux/mls/setrans.conf so that s1 maps to "Internal" and s2 maps to "Restricted". For this exercise, you can leave everything else as-is in the configuration file. Only change the values for s1 and s2.

```
...output omitted...
s1=Internal
...output omitted...
s2=Restricted
...output omitted...
```

5. Start and enable the mcstrans service.

```
[root@servera ~]# systemctl enable --now mcstrans
Created symlink ...output omitted...
```

6. Check the status of the mcstrans service.

```
[root@servera ~]# systemctl status mcstrans
...output omitted...
     Active: active (running) ...
...output omitted...
```

7. Check that s1 is mapped to the Internal category.

```
[root@servera ~]# chcat -L | grep Internal
s1                            Internal
```

8. Check that s2 is mapped to the Restricted category.

```
[root@servera ~]# chcat -L | grep Restricted
s2                            Restricted
```

9. Exit the SSH connection to finish the activity.

```
[root@servera~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

4.1. Filesystem and Process Management under MLS

Objective: Work with files, directories, and processes while adhering to MLS dominance rules.

Securing the Filesystem

SELinux Multi-Level Security (MLS) mode secures the filesystem by applying hierarchical sensitivity levels and non-hierarchical categories to every file and process. It enforces a modified Bell-LaPadula mandatory access control model, ensuring that subjects can only read and write data according to strict clearance boundaries.

The core components of MLS filesystem security are sensitivity, categories, and dominance. Sensitivity involves hierarchical levels of data classification (e.g., s0 up to s15, mapping conceptually to labels such as Unclassified to Top Secret). Categories are non-hierarchical compartments that are used to enforce the "need-to-know" principle (e.g., c0 for Finance, c1 for HR). Dominance ensures that a process can only access a file if its sensitivity level is equal to or higher than the file's level, and its category set contains all categories assigned to the file.

Elevating file sensitivity levels in MLS

By incorporating a localized SELinux policy module, you can permit Multi-Level Security users to upgrade file classification thresholds. Whenever an authorized user edits a file, its sensitivity rating is automatically elevated to match the lowest value within that user's active security clearance scope.

Standard configurations restrict Multi-Level Security (MLS) users from upgrading file classification thresholds. To bypass this limitation, the security administrator (secadm_r) can introduce the local mlsfilewrite module into the active system SELinux policy rules. Consequently, system users linked to the designated SELinux type within this module gain the ability to step up file sensitivity ratings simply by editing the resource. Each time an adjustment is made to a file, its sensitivity rating automatically scales up to match the minimum boundary of the user's current operational security clearance scope.

When operating within the authorized secadm_r administrative role, a security administrator can adjust specific file security boundaries directly by executing the chcon -l s0 /path/to/file command structure.

Changing file sensitivity in MLS

Administrators can manually elevate a file's MLS classification level to ensure it is processed at a higher sensitivity tier, which helps safeguard data against access by users with lower clearance levels.

The SELinux MLS policy restricts standard users to modifying files only at their specific sensitivity level. This mechanism is designed to prevent the exposure of highly sensitive information to lower-clearance users and to stop those with lower clearance from creating high-sensitivity materials. Administrators, however, have the authority to manually upgrade a file's classification level, enabling it to be processed at a higher security tier.

The process for changing file sensitivity in MLS is as follows:

View the file's classification level.

```
# ls -lZ /path/to/file
-rw-r-----. 1 User1 User1 user_u:object_r:user_home_t:s1 ...
```

Modify the default classification level of the file.

```
# semanage fcontext -a -t user_home_t -r s2 /path/to/file
```

Relabel the SELinux context of the file.

```
# restorecon -F -v /path/to/file

Relabeled /path/to/file from user_u:object_r:user_home_t:s1 to ...
```

Verify the classification level of the file.

```
# ls -lZ /path/to/file
-rw-r-----. 1 User1 User1 user_u:object_r:user_home_t:s2 ...
```

Check that the lower-clearance user is unable to read the file.

```
$ cat /path/to/file
cat: file: Permission denied
```

Process Execution under MLS

Under Multi-Level Security (MLS), process execution is strictly governed by data classification labels and mandatory access control policies to prevent the unauthorized disclosure or leak of sensitive data. Processes are treated as subjects (active entities) and files, memory, or devices are treated as objects (passive entities).

Process execution follows the Bell-LaPadula (BLP) model, which controls the flow of information through two primary rules: The Simple Security Property ("No Read Up") and the *-Property / Star-Property ("No Write Down").

With the Simple Security Property, a running process can only read data from an object if the object's security clearance level is equal to or lower than the process's own clearance level. For example, a process running at the Secret level can read files marked as Secret or Unclassified, but it is entirely blocked from reading any data marked as Top Secret.

With the *-Property / Star-Property, a running process can only write or append data to an object if the object's security level is equal to or higher than the process's clearance level. For example, a process running at the Secret level can write data up into a Top Secret file, but it is blocked from writing data down to an Unclassified file. This strict rule ensures that a process (or a rogue program like a Trojan horse) cannot accidentally or maliciously leak classified information into a lower-security space where unauthorized users could see it.

With security labels, every process is dynamically initialized with a specific clearance label containing a hierarchical sensitivity (e.g., Unclassified, Confidential, Secret, Top Secret) and a set of non-hierarchical categories or compartments (e.g., Finance, Projects).

There is a dominance requirement involved.: For a process to successfully access or interact with an object, the process's clearance level must dominate the object's level (meaning its hierarchical tier must be equal or higher, and its authorized compartments must completely encompass the object's compartments).

When a process spawns or executes a child process, the child process typically inherits the exact security label of the parent process to maintain a consistent environment. Also, certain operating system utilities (like backup routines or administrative tools) are designated as "Trusted Subjects". These specific processes are allowed to bypass the standard "No Write Down" rule to perform necessary tasks (like declassification or maintenance), but they require rigorous, high-assurance code vetting.

How processes inherit or transition sensitivity levels

Under Multi-Level Security (MLS), processes handle sensitivity levels through strict rules governing execution, inheritance, and transition. These mechanisms prevent data leaks from higher classification levels to lower ones.

By default, processes inherit their security attributes directly from their parent. Therefore, at the default level, a new process inherits the exact sensitivity level of the creating process. During user login, a user's initial shell inherits the clearance level selected at login. There is a boundary for clearance, where the inherited level must fit within the user's maximum assigned clearance.

Processes can change their sensitivity levels under strict architectural constraints. Only trusted, explicit system calls can alter a running process level. Also, processes cannot lower their level if they hold high-sensitivity data. Security policies in SELinux MLS define valid target transition states.

Running commands at specific levels using runcon

To run a command at a specific MLS level using runcon, you use the -l (or --range) option followed by the target sensitivity level and optional categories. You can either modify your current context incrementally or pass a fully specified security context string.

The recommended way is to modify the level incrementally, as in the example below (The example runs a script at sensitivity level 2 with category 5).:

```
$ runcon -l s2:c5 ./my_script.sh
```

Polyinstantiated Directories

To prevent data leaks through shared temporary spaces like /tmp or /var/tmp, MLS uses polyinstantiation. When processes with different MLS levels access a shared directory, the system transparently redirects them to separate, isolated sub-directories. A process at s1 and a process at s2 will see entirely different contents inside /tmp, preventing covert channels and data tracking.

Preventing information leaks in shared spaces (like /tmp and /var/tmp)

To safeguard against information disclosure and race-condition vulnerabilities, configure polyinstantiation for home directories as well as the /tmp and /var/tmp directories. This isolation strategy provisions individual, distinct mounts per user, mitigating risks tied to shared temporary repositories.

Standard system behavior dictates that all users, utilities, and services utilize /tmp, /var/tmp, and home directories as common temporary storage locations, rendering them susceptible to information leakage via file names and race conditions. Instantiating these directories ensures they are completely isolated among users, dynamically mapping specific mounts such as /tmp-inst and /var/tmp/tmp-inst to the traditional /tmp and /var/tmp locations. For additional technical details, consult the documentation provided in /usr/share/doc/pam-docs/txts/README.pam_namespace, which is bundled with the pam-docs package.

The procedure to configure and use polyinstantiation is as follows:

Turn on polyinstantiation using an SELinux boolean.

```
# setsebool -P allow_polyinstantiation 1
```

For data persistence over reboot, build the directory structure with the required permissions.

```
# mkdir /tmp-inst /var/tmp/tmp-inst --mode 000
```

Restore the security context on the directory structure.

```
# restorecon -Fv /tmp-inst /var/tmp/tmp-inst
Relabeled /tmp-inst ...
Relabeled /var/tmp/tmp-inst ...
```

Permit the fapolicyd application control framework to track file access occurrences on the host file system if it is deployed on your system. Do this by enabling the allow_filesystem_mark option in the /etc/fapolicyd/fapolicyd.conf configuration file.

```
allow_filesystem_mark = 1
```

Activate polyinstantiation across the /tmp, /var/tmp/, and user home directories. Important: You must utilize /etc/security/namespace.conf rather than creating a separate file inside the /etc/security/namespace.d/ directory, because the pam_namespace_helper application does not parse additional files within /etc/security/namespace.d/

On a system with multi-level security (MLS), uncomment the last three lines in the /etc/security/namespace.conf file.

```
/tmp      /tmp-inst/          level     root,adm
/var/tmp  /var/tmp/tmp-inst/  level     root,adm
$HOME     $HOME/$USER.inst/   level
```

Check that the pam_namespace.so module is properly configured for the session.

```
$ grep namespace /etc/pam.d/login
```

To enable polyinstantiation for a specific service, insert the session required pam_namespace.so unmnt_remnt directive into its module configuration immediately following the session include system-auth line. For example, in /etc/pam.d/su, /etc/pam.d/sudo, /etc/pam.d/ssh, and /etc/pam.d/sshd:

```
...
session         include         system-auth
session         required        pam_namespace.so unmnt_remnt
```

To ensure these updates are applied, any user active prior to the polyinstantiation configuration must sign out and log back in. Afterward, sign in using a non-root account and confirm that the /tmp/ directory has been successfully mounted to /tmp-inst/.

```
$ findmnt --mountpoint /tmp/
TARGET    SOURCE                          FSTYPE    OPTIONS
/tmp      /dev/vda1[/tmp-inst/<user>    xfs       rw ...
```

4.2. Guided Exercise: Filesystem and Process Management under MLS

In this exercise, you will manage user clearance to ensure that they cannot read above or write below their clearance.

1. Connect via SSH to servera as the sysadmin user.

```
student@workstation:~$ ssh sysadmin@servera
```

2. Elevate privileges to become the root user.

```
[sysadmin@servera ~]$ sudo -i
[sudo] password for sysadmin: redhat
[root@servera ~]#
```

3. Create the operator1 user.

```
[root@servera ~]# useradd operator1
```

4. Set the password for the operator1 user to redhat.

```
[root@servera ~]# passwd operator1
New password: redhat
Retype new password: redhat
passwd: password updated successfully
```

5. Logout of the current ssh session.

```
[root@servera ~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

6. Connect via SSH as the operator1 user to servera.

```
student@workstation:~$ ssh operator1@servera
```

7. Show the security context of the operator1 user.

```
[operator1@servera ~]$ id -Z
user_u:user_r:user_t:s0
```

8. Create a text file named file1 that contains the line “This is file1!”.

```
[operator1@servera ~]$ echo "This is file1!" > file1
```

9. View the security context of the file1 file.

```
[operator1@servera ~]$ ls -Z file1
user_u:object_r:user_home_t:s0 file1
```

10. Check that the operator1 user can access the content of file1.

```
[operator1@servera ~]$ cat file1
This is file1!
```

11. Exit the SSH session.

```
[operator1@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

12. SSH as the sysadmin user to servera.

```
student@workstation:~$ ssh sysadmin@servera
```

13. Elevate privileges to become the root user.

```
[sysadmin@servera ~]$ sudo -i
[sudo] password for sysadmin: redhat
[root@servera ~]#
```

14. View the file1 file’s classification level.

```
[root@servera ~]# ls -lZ /home/operator1/file1
-rw-r--r--. 1 operator1 operator1 user_u:object_r:user_home_t:s0
```

15. Modify the classification level of the file to s2.

```
[root@servera ~]# semanage fcontext -a -t user_home_t \
-r s2 /home/operator1/file1
```

16. Relabel the SELinux context of the file.

```
[root@servera ~]# restorecon -Fv /home/operator1/file1
Relabeled ...output omitted...
```

17. Verify the classification level of the file.

```
[root@servera ~]# ls -lZ /home/operator1/file1
-rw-r--r--. operator1 operator1 system_u:object_r:user_home_t:s2
```

18. Exit the SSH session.

```
[root@servera ~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

19. SSH to servera as the operator1 user.

```
student@workstation:~$ ssh operator1@servera
```

20. Check that the lower-clearance operator1 user is now unable to read the file.

```
[operator1@servera ~]$ cat file1
cat: file1: Permission denied
```

21. Exit the SSH session to end the activity.

```
[operator1@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

5. Network Configuration and Services in MLS

Objective: Secure network traffic and configure system services to handle multi-level data securely.

MLS and Network Security

SELinux Multi-Level Security (MLS) mode secures network traffic by assigning hierarchical sensitivity levels (such as Confidential or Top Secret) and non-hierarchical categories to network sockets, ports, and data packets, enforcing strict Bell-LaPadula mandatory access control rules that prevent unauthorized data flow across security domains.

To govern how processes send, route, and receive data packets, network traffic utilizes a combination of security contexts and policy rules. Every network packet receives a security label based on its source, destination, and associated processes. The SELinux policy subsequently checks these labels against predetermined rules to authorize or reject the communication.

The core network security mechanisms in MLS are as follows:

Packet and Socket Labeling: SELinux tags network sockets and outgoing packets with precise security contexts, binding data streams to distinct classification tiers.

Sensitivity Hierarchies: Labels incorporate ranks such as Unclassified, Confidential, Secret, and Top Secret to evaluate the clearance of both the sending process and the receiving endpoint.

Dominance Restrictions: Communication requires that the sender and receiver clearances dominate or match specific constraints, stopping lower-level processes from accessing high-level ports and vice versa.

Port Type Enforcement: Network ports are assigned distinct types, restricting services bound to those ports to communicating only with explicitly permitted clearance levels.

Introduction to NetLabel and CIPSO (Commercial IP Security Option)

Network traffic in SELinux can be labeled using two primary methods: IPsec-based labeling and NetLabel, which serves as an implementation of CIPSO. NetLabel is particularly beneficial

for facilitating cooperation between SELinux and legacy trusted operating systems, such as Trusted HP-UX and Trusted Solaris, because it solely supports the MLS component of the SELinux context.

NetLabel manages network security contexts in SELinux by mapping Multi-Level Security (MLS) sensitivity levels and categories onto outgoing network packets using protocols like CIPSO for IPv4, allowing systems to enforce mandatory access controls across the network. NetLabel works using a combination of packet marking, traffic reception, and access control enforcement.

With packet marking, NetLabel intercepts outgoing network traffic from a process while adding a DOI (Domain of Interpretation) and CIPSO/CALIPSO option headers containing the MLS sensitivity level and categories.

To handle traffic reception, the NetLabel subsystem extracts the security attributes and assigns a peer label (netlabel_peer_t) to the socket connection. This happens when an incoming packet arrives with a CIPSO tag.

For access control enforcement, the SELinux policy checks if the receiving process domain has authorization to receive data from the translated network peer label before allowing communication.

Labeling network packets and network interfaces

In SELinux Multi-Level Security (MLS), network interfaces are labeled using the semanage interface command to assign an MLS-enabled type and sensitivity range, while network packets are labeled dynamically using SECMARK (via iptables/nftables) or peer labeling frameworks like NetLabel/CIPSO and Labeled IPSec.

You use the semanage tool to manage network interface types and bind MLS ranges to physical or logical cards.

Below is an example of how to add or update an interface type.:

```
# semanage interface -a -t netif_t -r s0:c0.c10alet eth0
```

For CIPSO-based networks, configure remote hosts and local NetLabel subsystems to translate wire-level DOI (Domain of Interpretation) category and sensitivity tags directly into incoming socket and packet MLS contexts.

When using Labeled IPSec, associate Security Associations (SAs) in policies with explicit MLS labels so that traffic arriving from a peer instantiates with the correct remote clearance level.

Securing Common Services

Configuring OpenSSH for MLS logins (handling user transitions)

To configure OpenSSH for SELinux Multi-Level Security (MLS) logins and handle proper user and range transitions, you must ensure PAM is enabled in OpenSSH, map system user identities correctly using semanage login, and allow proper dynamic context transitions via pam_selinux.so.

To verify OpenSSH and PAM configuration, open /etc/ssh/sshd_config and verify PAM support is active.

```
UsePAM yes
```

Also, ensure that keyboard-interactive or password/certificate mechanisms pass environment variables required by SELinux. Keep the PermitUserEnvironment parameter disabled to prevent unauthorized context override.

```
PermitUserEnvironment no
```

Then, define the mapping between Linux login names and SELinux user identities with specific sensitivity levels or clearance ranges using semanage.

```
# semanage login -a -s staff_u -r s0-s15:c0.c1023 username
```

To set up default fallback contexts for unmapped users in /etc/selinux/mls/seusers or via the default login mapping, do the following.

```
# semanage login -m -s user_u -r s0 __default__
```

To validate transitions, check audit logs at /var/log/audit/audit.log during initial login tests to catch any dyntransition or avc: denied errors.

Multilevel-aware services vs. single-level restricted services

Multilevel-aware services operate across a range of security sensitivities and dynamically adjust their access based on client clearance levels, while single-level restricted services are locked to a fixed, rigid security label and cannot cross sensitivity boundaries.

Multilevel-aware services are assigned a clearance range (e.g., s0-s15) allowing them to process requests from multiple hierarchical sensitivities (such as Unclassified up to Top Secret). These services can dynamically change or adopt the specific label of the active client session to read or write data at that exact level. Built explicitly to understand sensitivity labels, category compartments, and Bell-LaPadula constraints, multilevel-aware services are complex in design. Some examples are trusted database servers or multilevel mail cross-domain guards that handle inputs from varied tiers.

Single-level restricted services are bound to a precise, single security level (e.g., strictly s0 or s0:c0.c5) with no clearance range. These services are blocked from interacting with any data, files, or clients outside of that exact single level, enforcing compartment isolation. Simpler in implementation, these are treated by the kernel as a standard confined subject that cannot alter its operational sensitivity. Some examples are a dedicated daemon processing only unclassified public web traffic or a localized utility bound persistently to a low-level compartment.

Trusted Programs

In SELinux Multi-Level Security (MLS), specific programs are designated as "trusted" with a mls_trustedrange attribute or explicit policy exceptions. They bypass strict Bell-LaPadula constraints (like reading down or writing only at exact levels) to perform necessary cross-level tasks, such as authentication, logging, or setting initial user session ranges.

Understanding programs that can bypass MLS constraints (e.g., sshd, login)

There are several examples of programs or utilities that can bypass MLS constraints.

- login / getty: Authenticates users and transitions the session from a system-wide or neutral context to the user's specific clearance range.

- sshd (OpenSSH daemon): Accepts remote connections across varying network clearance levels and negotiates or assigns the proper MLS range upon successful user authentication.

- cron / anacron: Scheduled task utilities that often need trusted range permissions to execute background jobs on behalf of users across different sensitivity levels.

- newrole: A specialized utility that allows a user to change their current security level or role within their authorized clearance bounds.

Trusted programs operate using a defined sensitivity range (e.g., s0-s0:c0.c1023) rather than a single fixed level, allowing them to bridge lower and higher clearance data streams. Policies use specific rules (such as mlsconstrain overrides) granting explicit privileges so utilities can write or read outside strict single-level boundaries during session initialization.

6.1. Troubleshooting, Auditing, and Log Analysis

Objective: Identify, interpret, and resolve AVC denials specific to MLS constraints.

Analyzing MLS Auditing Logs

When enabling SELinux on systems where it was previously disabled or when operating services in non-standard setups, you may encounter issues blocked by security policies that require troubleshooting. It is worth noting that SELinux denials typically stem from system misconfigurations.

Deciphering MLS-specific fields in /var/log/audit/audit.log

Identify SELinux issues by searching audit logs for Access Vector Cache (AVC) denial messages. Reviewing AVC logs helps quickly determine whether SELinux is preventing an operation.

When SELinux blocks an operation, inspect /var/log/audit/audit.log first for additional details on the denial. Use the ausearch utility to query the audit logs.

The following steps can be taken to identify problems related to SELinux.

Because SELinux caches access decisions—such as granting or denying permissions—in the Access Vector Cache (AVC), specify AVC and USER_AVC as the message type parameters.

```
# ausearch -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent
```

In Linux Security-Enhanced (SELinux) Multi-Level Security (MLS) environments, auditd records context labels via subj= (subject) and obj= (object) fields inside log files like /var/log/audit/audit.log. These sensitivity levels and compartments follow the format sensitivity:category.

Below are some things to consider when deciphering MLS context formats.

Subject/Object Labels: s0-s15:c0.c1023 where s0 to s15 represent hierarchical sensitivity levels, and c0.c1023 represent non-hierarchical category compartments.

Range Formats: s0 (low single level) vs s0-s15:c0.c255 (a clearance range from minimum to maximum bounds).

Let's consider this sample SELinux MLS denial record in /var/log/audit/audit.log.:

```
type=AVC msg=audit(1658312456.789:123): avc: denied { read } for
pid=4512 comm="cat" name="secret.txt"
obj=unconfined_u:object_r:user_home_t:s15:c5,c10
subj=staff_u:staff_r:staff_t:s0:c1
```

We can break down the MLS fields to make sense of the log details.

- subj=staff_u:staff_r:staff_t:s0:c1
  - **Sensitivity:** s0 (System low / lowest clearance level).
  - **Category:** c1 (Assigned compartment/category 1
- obj=unconfined_u:object_r:user_home_t:s15:c5,c10
  - **Sensitivity:** s15 (Top/High sensitivity level).
  - **Categories:** c5,c10 (Strict compartments required to access the object).

The verdict: The access is denied because the subject's sensitivity (s0) is lower than the object's required sensitivity (s15), and compartments do not match (Bell-LaPadula dominance rules fail).

Identifying "Write Down" or "Read Up" violations

To find SELinux Multi-Level Security (MLS) "read up" or "write down" violations in /var/log/audit/audit.log, search for type=AVC records containing denied where the source context (scontext) and target context (tcontext) sensitivity/range levels violate Bell-LaPadula constraints (no read up, no write down). Use ausearch or grep to isolate these events.

Below are steps to find violations via command line tools.

Filter audit logs for AVC denials using the audit search tool.

```
# ausearch -m AVC -i
```

Search specifically for denied keywords inside the log.

```
# grep "avc:  denied" /var/log/audit/audit.log
```

Pipe results to examine specific clearance range mismatches in contexts.

```
# grep "denied" /var/log/audit/audit.log | \
grep -E 's[0-9]+:c.*s[0-9]+:c'
```

An MLS denial entry displays scontext (subject/process level) and tcontext (target/object level).

For a read up violation example: A process running at a lower sensitivity (e.g., s0) attempts to read a file at a higher sensitivity (e.g., s15 or Secret), triggering a denial for a permission like { read } or { getattr }.

For a write down violation example: A process running at a higher sensitivity (e.g., s15) attempts to write or create an object at a lower sensitivity (e.g., s0 or SystemLow), triggering a denial for permissions like { write }, { create }, or { add_name }.

Troubleshooting Tools

By utilizing available troubleshooting utilities, administrators can effectively examine SELinux access denials, pinpoint root causes, and separate policy constraints from general system misconfigurations.

Using ausearch, aureport, and sealert

As long as the policycoreutils-python-utils and setroubleshoot-server packages are installed on your system, you can use ausearch, aureport, and sealert to analyze SELinux denial messages.

As mentioned before, the ausearch command can be used to query the audit logs.

```
# ausearch -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent
```

To run a general summary of failed events to spot trends, use this aureport command.

```
# aureport -m --failed --summary -i
```

To check for SELinux-specific anomaly and error reports, you can use the --avc option with aureport.

```
# aureport --avc --summary -i
```

To obtain additional detailed information regarding a recorded denial, execute the sealert command.

```
# sealert -l "*"
```

Writing custom policy modules using audit2allow (and when not to use them in an MLS environment)

Although Red Hat advises against using custom SELinux policy modules to avoid unsupported configurations, introducing custom rules can occasionally be required—such as allowing an application to interact with assets in non-standard file paths.

Circumstances where building a custom policy module is suitable include:

- Resolving a documented bug (like a missing policy rule), following the recommendations provided in the relevant Knowledge Base article.
- Deliberately customizing an installation for an unhandled scenario (such as permitting mysqld to reach files hosted on a remote file system).

You should avoid writing a custom policy module under the following conditions:

- audit2allow identifies that files have incorrect labels
- audit2allow determines the necessary feature can be enabled via a boolean
- You suspect an underlying flaw or bug exists within the policy
- audit2allow generates an excessively long list of rules for a specific SELinux type, suggesting a broader underlying issue
- audit2allow recommends rules involving write, append, or create operations on default_t or usr_t contexts

Red Hat includes audit2allow in the policycoreutils-python-utils package. This utility automatically generates proposed policy rules based strictly on audit log entries.

Avoid relying on this tool to build custom policy modules, as modifying your system's security configuration requires a thorough understanding of the underlying implications of each added rule.

Its recommended application is limited to running audit2allow -a to quickly review failures and gain insight into active issues.

The recommended process involves the following steps.

First, collect AVCs using both the ausearch and audit2allow utilities.

The following command demonstrates how to display Access Vector Cache (AVC) denials in an easily readable format, making it straightforward to analyze what resources a failing application is trying to access and how it operates.

```
# ausearch -m avc,user_avc
```

Then, to print rules that would avoid having AVCs if they were applied to the system, use this audit2allow command.

```
# audit2allow -a -t <selinux_type>
```

Because the output generated by audit2allow lacks comprehensive details, relying on it without prior context can easily lead policy authors to misinterpret the true underlying cause of an issue. Consequently, utilizing this tool is strongly advised only after thoroughly examining and analyzing the results from the ausearch command.

Next, we create a text file that has the .te file extension. This file contains the policy to build.

Below is a template that explains the format of the policy file.

```
policy_module(local_<type_or_daemon>, <version>)

gen_require(`
    type <type>;
    ...
')

<rules>
```

Once the file is completed with real policy details, we compile the policy module.

```
# make -f /usr/share/selinux/devel/Makefile \
local_<type_or_daemon>.pp
```

And for the final step, we install the policy module.

```
# semodule -i local_<type_or_daemon>.pp
```

Emergency Recovery

To recover a broken Multi-Level Security (MLS) SELinux system that prevents a normal boot, interrupt the GRUB bootloader, append enforcing=0 or selinux=0 to the kernel command line, boot into a permissive environment, mount the root file system, correct the misconfigured policy or labels, and trigger a full relabel.

Booting into selinux=0 or enforcing=0 to recover a broken MLS system

The steps to recover a broken MLS system are as follows.:

- Restart your machine and wait for the GRUB boot loader menu.
- Highlight your normal boot entry and press e to edit the parameters.
- Find the long line starting with linux.
- Append enforcing=0 (to switch to permissive mode) or selinux=0 (to completely disable SELinux checks/loading for that boot) to the end of that line.
- Press Ctrl + X or F10 to boot using these temporary settings.

Then, access and repair the system.

- Log in as the root user once the emergency or standard prompt appears.

- If you used rd.break or booted into a read-only rescue environment, remount your root file system as read-write:

```
# mount -o remount,rw /sysroot
# chroot /sysroot
```

Open your SELinux configuration file (usually /etc/selinux/config) to verify that SELINUXTYPE=mls is set correctly and change SELINUX=enforcing to SELINUX=permissive so you can safely test the system on the next reboot without getting locked out.

Investigate or reverse the broken policy changes, mislabeled files, or bad security contexts causing the boot failure.

Once finished with the steps to reverse the broken changes, trigger a relabel and restore enforcement.

- Create an autorelabel trigger file in the root directory so the system rebuilds all complex MLS sensitivity labels correctly on startup:

```
# touch /.autorelabel
```

- Exit the chroot environment (if applicable) and reboot the system normally:

```
# reboot
```

- Allow the relabeling process to finish completely (this can take time depending on your drive size).

- Once back up in permissive mode with a working system, edit /etc/selinux/config back to SELINUX=enforcing and run setenforce 1 or reboot one final time to secure the MLS environment.

6.2. Guided Exercise: Troubleshooting, Auditing, and Log Analysis

In this exercise, you will troubleshoot an SSH connection failure caused by a MLS range mismatch.

1. Connect via SSH to servera as the sysadmin user.

```
student@workstation:~$ ssh sysadmin@servera
```

2. Elevate privileges to become the root user.

```
[sysadmin@servera ~]$ sudo -i
[sudo] password for sysadmin: redhat
[root@servera ~]#
```

3. Create the sysadmin2 user.

```
[root@servera ~]# useradd sysadmin2
```

4. Set the password for the sysadmin2 user to redhat.

```
[root@servera ~]# passwd sysadmin2
New password: redhat
Retype new password: redhat
passwd: password updated successfully
```

5. Map the sysadmin2 user to the sysadm_u SELinux user.:

```
[root@servera ~]# semanage login -a -s sysadm_u -r \
"s0-s15:c0.c1023" sysadmin2
```

6. Exit the SSH session.

```
[root@servera ~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

7. Attempt to SSH as the sysadmin2 user to servera.

```
student@workstation:~$ ssh sysadmin2@servera
Read from remote host servera: Connection reset by peer
Connection to servera closed.
client_loop: send disconnect: Broken pipe
```

8. Connect via SSH to servera as the sysadmin user.

```
student@workstation:~$ ssh sysadmin@servera
```

9. Elevate privileges to become the root user.

```
[sysadmin@servera ~]$ sudo -i
[sudo] password for sysadmin: redhat
[root@servera ~]#
```

10. Filter audit logs for AVC denials using the audit search tool. Look for messages related to the sysadmin2 user, the sysadm_u SELinux user, or dyntransition. A dyntransition error in SELinux MLS mode during an SSH connection means the SSH daemon (sshd) is blocked from dynamically changing its process security context or range to match the user's authorized clearance level. SELinux prevents this state shift because the active policy lacks permission for that specific transition.

```
[root@servera ~]# ausearch -m AVC | grep ssh
...output omitted...
type=AVC msg=audit(1786672266.801:39076): avc: denied { dyntransition
} ... sshd-session ... tcontext=sysadm_u:sysadm_r:sysadm_t:s0-
s15:c0.c1023 ...
```

11. To fix this dyntransition error, which is caused by an MLS range mismatch, we should map the sysadmin2 user to the staff_u SELinux user. This enforces the principle of least privilege, allowing the user to log in safely as a regular user while retaining a secure path to elevate to an administrative role via sudo.

```
[root@servera ~]# semanage login -m -s staff_u -r \
"s0-s15:c0.c1023" sysadmin2
```

12. Use the visudo command to edit the sudoers file.

```
[root@servera ~]# visudo
```

13. Add the entry in bold to the sudoers configuration file (in the location shown) and then save.

```
...output omitted...
## Allow root to run any commands anywhere
root      ALL=(ALL)      ALL

## Allow secadmin to run commands as root under the secadm_r role
secadmin ALL=(ALL)  TYPE=secadm_t  ROLE=secadm_r  ALL

## Allow sysadmin to run commands as root under the sysadm_r role
sysadmin ALL=(ALL) TYPE=sysadm_t   ROLE=sysadm_r  ALL

## Allow sysadmin2 to run commands as root under the sysadm_r role
sysadmin2 ALL=(ALL) TYPE=sysadm_t  ROLE=sysadm_r ALL
```

14. Verify that sysadmin2 is mapped to the staff_u SELinux user.

```
[root@servera ~]# semanage login -l
Login Name     SELinux User    ...
...output omitted...
sysadmin2      staff_u         ...
```

15. Exit the SSH session.

```
[root@servera ~]# exit
exit
[sysadmin@servera ~]$ exit
logout
Connection to servera closed.
student@workstation:~$
```

16. Make a successful SSH connection as the sysadmin2 user to servera.

```
student@workstation:~$ ssh sysadmin2@servera
```

17. Elevate privileges to become the root user.

```
[sysadmin2@servera ~]$ sudo -i
[sudo] password for sysadmin2: redhat
[root@servera ~]#
```

18. Verify that the login is mapped to the sysadm_r role.

```
[root@servera ~]# id -Z
staff_u:sysadm_r:sysadm_t:s0-s15:c0.c1023
```

19. Exit the SSH connection to end the activity.

```
[root@servera ~]# exit
exit
[sysadmin2@servera ~]$ exit
logout
Connection to servera closed.
```